



Fast Food Restaurant Database System

By Bo Zhang, Weitong Ding, Xiaotong Luo, Aaron Osterman
Instructor: Dr. Romila Pradhan
Date Submitted: April 18, 2025

Company Description: Fast Food Restaurant



Local fast food restaurants

Service Options:

Eat here, take-out and drive-through

Menu Highlights:

Burgers, fries, chicken sandwiches, milkshakes

Modernization goals:

Implement a database system to manage:

Customer data and order history

Employee role and task assignment

Kitchen scheduling

Foreground operation

Database user:

Cashiers, cooks, managers, customers

Business Rules



A customer can place multiple orders.

A customer has a unique ID, name, email, phone number, date of birth, membership tier, and reward points.

An order is placed by one customer only.

Each order has a unique ID and order time.

Payment is made when the order is placed.

An order is handled by one kitchen only.

A kitchen can handle multiple orders.

Business Rules



A kitchen has a unique ID, location, capacity, and operating hours.

An employee has a unique ID, name, SSN, DOB, phone number, and email.

Employees can be a Cashier, Chef, or Manager.

A kitchen can have zero or more chefs.

A chef can work in one kitchen only.

An order has a status

Orders are picked up from a front counter.

Business Rules



One order is linked to one front counter.

A front counter can hold multiple orders.

Each front counter has an ID, location, and capacity.

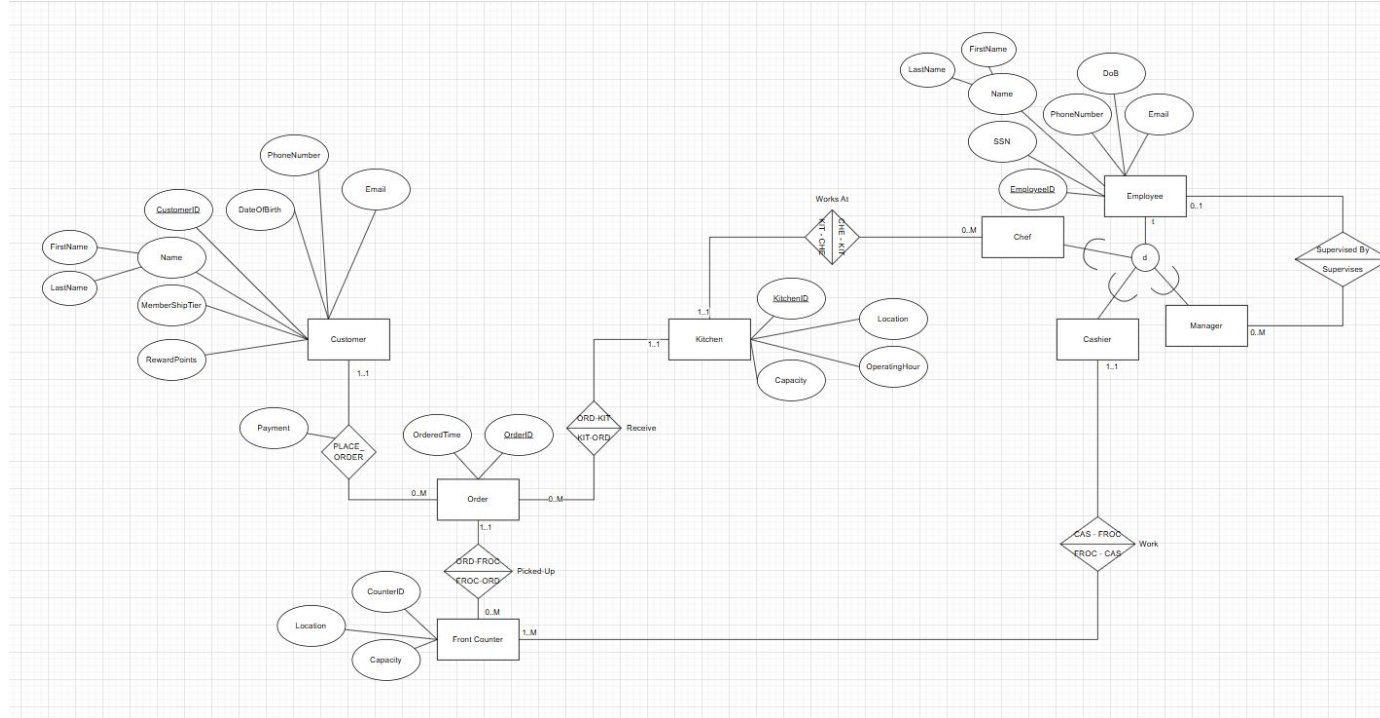
Each counter has one or more cashiers.

One cashier is assigned to exactly one counter.

A manager can supervise multiple employees.

An employee may be supervised by zero or one manager.

EER Diagram



Relational Data Model



CUSTOMER (CUSTOMERID, FIRSTNAME, LASTNAME, DATE_OF_BIRTH, PHONE_NUMBER, EMAIL, MEMBERSHIP_TIERS, REWARD_POINTS)

ORDERS (ORDERID, ORDER_TIME, STATUS, *CUSTOMERID*, *KITCHENID*)

- NOTE: Foreign keys, *KITCHENID* are NOT NULL.

PLACE_ORDER (*CUSTOMERID*, *ORDERID*, PAYMENT)

KITCHEN (KITCHENID, LOCATION, CAPACITY, OPERATING_HOUR)

FRONT_COUNTER (COUNTERID, COUNTER_LOCATION, CAPACITY, *ORDERID*, *EMPLOYEEID*)

- NOTE: Foreign keys *ORDERID*, *EMPLOYEEID* cannot be NULL

EMPLOYEE (EMPLOYEEID, SSN, FIRSTNAME, LASTNAME, DATE_OF_BIRTH, EMAIL, PHONE_NUMBER, SUPERVISORID)

CHEF (*EMPLOYEEID*, *KITCHENID*)

- NOTE: *KITCHENID* cannot be NULL

CASHIER (*EMPLOYEEID*)

MANAGER (*EMPLOYEEID*)



Comparison (Benefits & Drawbacks)

Benefits

1. In the aspect of business, the relational data model ensures the data consistency and integrity between different entities since they relate to primary keys and foreign keys. For example, in the traditional business model, the information of order might get wrong in the process of transferring between customer, front counter, and kitchen, but now the data model can ensure the data will stay the same.
2. The relational data model provides convenience for the business to analysis their data. Users can get the data they need easily by using language such as SQL from the database.

Drawbacks

1. If there is some modification to the business, such as if the restaurant wants to provide delivery services, then it will be a bit hard to update the data schema.
2. Since each table has their relationship to other tables, as the time goes by, large amounts of data might influence the performance of the database.

Physical Design Overview



Database Type: Relational DBMS (RDBMS)

Index Structure Used: B-Tree

Chosen for efficiency in read/write operations

Design Goals:

Optimize performance for frequent queries

Maintain data integrity via relationships and constraints

Ensure scalability for future features (e.g., delivery)

Key Aspects:

Primary keys for unique record identification

Foreign keys to maintain relational consistency

Secondary indexes on frequently queried fields

Indexing Strategy



Table	Indexed Field	Purpose
Customer	CUSTOMERID (PK)	Quick lookup of customer records
Employee	EMPLOYEEID (PK), SUPERVISORID	Retrieve employee info and their manager easily
Kitchen	KITCHENID (PK)	Identify kitchen capacity and status
Orders	ORDERID (PK), CUSTOMERID, KITCHENID, ORDER_TIME	Track and filter orders by customer, kitchen, or time
Place_Order	(CUSTOMERID, ORDERID) (PK)	Efficient join and search for placed orders
Front_Counter	COUNTERID (PK), ORDERID, EMPLOYEEID	Help locate orders or staff at specific counters
Chef/Cashier	EMPLOYEEID (PK)	Employee specialization indexing



Representative Queries

1. Order Related

e.g. List all orders within a specific order period

2. Customer Related

e.g. List all orders based on customerID

3. Kitchen Related

e.g. List all Opened Orders received by the specific kitchen

4. Human Resources Related

e.g. List Employee's info based on their supervisor

SQL Script Highlights



E.g. To check the info of a daily orders, our SQL script will be like:

```
SELECT ORDERID, COUNT(*) AS numOfOrders, SUM(payment) AS totalPayment  
FROM PLACE_ORDER  
WHERE DATE(ORDER_TIME) = 2025-04-19  
GROUP BY ORDERID;
```

Query Optimization & Physical Design Impact



The physical design of the database we used can significantly affect the performance of our queries, from the perspective of effectiveness, convenience, as well as maintainability.

1. Index---Effectiveness
2. Foreign keys---Convenience
3. Maintainability

Conclusion



The database system improves the efficiency and accuracy of the fast food restaurant operation.

Our relational model ensures that:

Data integrity is achieved with clearly defined keys and constraints

Scalability for future service extensions

Use policy indexes for performance optimization

The system supports multiple user roles:

Cashiers, cooks, managers and customers (indirect)

Both the physical design and the representative queries are tailored to reflect real business requirements and ensure practical usability.